



DevOps와 AI

CI/CD · Docker · Kubernetes · Helm에 AI가 들어오는 방식

트렌드 소개

오늘 소개할 네 가지

1. CI/CD

코드를 작성하고 검토하는 단계에 AI가 들어온 경우

2. Docker

AI 모델 자체를 컨테이너로 패키징·배포하는 경우

3. Kubernetes

AI 모델을 클러스터 위에서 전문적으로 서빙하는 도구들

4. Helm

AI 어시스턴트가 Helm Chart 작성을 도와주는 경우

PART 1

CI/CD 단계의 AI

코드를 검토하고 테스트를 만드는 과정이 달라지고 있습니다



AI 코드 리뷰

사람이 보기 전에 AI가 먼저 한 번 훑어봅니다



어떻게 동작하나

개발자가 코드를 올리면(Pull Request), AI가 자동으로 그 코드를 읽고 버그·보안 문제·스타일 이슈를 코멘트로 남깁니다. 사람 리뷰어는 AI가 이미 걸러낸 것 위에서 더 중요한 판단(설계, 방향성)에 집중합니다.



GitHub Copilot Code Review, CodeRabbit, Claude Code Review 같은 도구들이 대표적입니다



왜 필요해졌나 — 요즘은 코드 자체를 AI가 많이 짜다 보니, 검토할 양도 같이 늘어났습니다

참고 다만 코멘트가 너무 많으면 오히려 무시당하는 부작용도 있어, 중요한 것만 남기도록 튜닝하는 것이 실무의 과제입니다.

AI 테스트 자동 생성

테스트 코드를 사람이 일일이 안 써도 됩니다



어떻게 동작하나

AI가 코드를 분석해서, 아직 테스트되지 않은 부분을 찾아 자동으로 테스트 코드를 만들어 줍니다. 화면 클릭 순서를 사람 말로 설명하면, 그걸 실제 실행 가능한 테스트 코드로 바꿔주는 도구도 있습니다.



GitHub Copilot의 테스트 생성 기능, TestSprite, QA Wolf 같은 도구가 대표적입니다



자연어(사람 말)로 "로그인 버튼을 누르면 홈 화면으로 이동해야 한다"라고 설명하면 테스트 코드가 나옵니다

핵심 실행·인프라 관리는 여전히 사람 몫입니다 — AI는 '테스트 코드를 짜는 시간'을 줄여주는 것이지, 테스트 전체를 대신하는 것은 아닙니다.

PART 2

Docker로 AI 모델 배포하기

우리가 웹페이지를 넣었던 것처럼, AI 모델도 컨테이너에 넣습니다



AI 모델도 결국 컨테이너입니다

우리가 했던 것과 원리는 완전히 같습니다



우리 프로젝트 vs AI 모델 배포

우리는 FROM nginx 이미지 위에 index.html을 올려서 이미지를 만들었습니다.

AI 모델 배포도 똑같습니다 — 다만 index.html 자리에 '모델 실행 프로그램 + 모델 파일'이 들어갑니다. 이미지가 훨씬 크고(수 GB~수십 GB), GPU가 필요하다는 점만 다릅니다.

실제로 쓰이는 도구 두 가지

둘 다 '모델을 컨테이너로 실행'해 주는 도구입니다



Ollama

개인 PC나 서버에서 가볍게 LLM을 돌릴 때 씁니다.
docker run으로 엔진을 먼저 띄운 뒤, ollama pull 명령으로 원하는 모델을 따로 받습니다. 설치와 사용이 간단해서 입문용으로 많이 씁니다.

<https://github.com/ollama/ollama>



vLLM

실제 서비스처럼 많은 사람이 동시에 요청을 보내는 상황에 맞춰 만들어진, 훨씬 빠르고 효율적인 서빙 엔진입니다. --model 옵션에 이름만 주면 모델을 자동으로 받아 바로 서빙을 시작합니다.

<https://vllm.ai/>

정리 모델을 받는 방식(따로/자동)은 다르지만, docker run으로 실행하고 포트를 열어 접속하는 전체 흐름은 우리가 한 것과 같습니다.

PART 3

Kubernetes 위에서 AI 모델 서빙하기

그냥 Deployment로도 되지만, 전문 도구를 쓰는 이유가 있습니다



KServe

쿠버네티스 표준에 가장 가까운 도구



가볍고 표준화된 서빙

CNCF(쿠버네티스를 관리하는 재단)의 공식 프로젝트입니다 (2025년 9월 Incubating 단계로 승격).

일반 Deployment로는 없는 기능들을 자동으로 제공합니다 — 트래픽이 없을 때 자동으로 0개까지 줄이기(비용 절감), 여러 모델 프레임워크(PyTorch, TensorFlow 등)를 같은 방식으로 다루기 같은 것들입니다.

Kubeflow

모델 서빙뿐 아니라 학습까지 아우르는 큰 플랫폼



AI 개발 전체 과정을 위한 플랫폼

사실 KServe는 원래 Kubeflow의 일부였다가 독립한 프로젝트입니다.

Kubeflow는 모델을 서빙하는 것뿐 아니라, 데이터 준비 → 모델 학습 → 실험 관리 → 배포까지, AI 모델이 만들어지는 전체 과정을 쿠버네티스 위에서 관리하는 훨씬 큰 플랫폼입니다.

Seldon Core

여러 모델을 복잡하게 엮을 때 강한 도구



복잡한 파이프라인과 설명 가능성

모델 하나만 서빙하는 게 아니라, 여러 모델을 순서대로 연결하거나(예: 전처리 모델 → 예측 모델 → 후처리) 조합해야 할 때 강합니다.

의료·금융처럼 '왜 이런 결과가 나왔는지' 설명이 꼭 필요한 업종에서 특히 많이 씁니다.

세 도구, 한눈에 비교

정답은 없고, 상황에 따라 다릅니다

	가장 강한 점	적합한 상황
KServe	가볍고 표준화됨	단일 모델을 빠르게 배포
Kubeflow	학습부터 배포까지 전체 관리	AI 개발 전체 파이프라인
Seldon Core	복잡한 파이프라인, 설명 가능성	여러 모델 연동, 규제 산업

공통점 셋 다 우리가 배운 쿠버네티스(Pod, Deployment, Service) 위에서 동작합니다 — 오늘 배운 기초가 있어야 이해할 수 있는 도구들입니다.

PART 4

Helm 작성을 돕는 AI

values.yaml, templates를 AI가 같이 써줍니다



AI가 Helm 작성을 돕는 방식

말로 설명하면 YAML을 만들어 줍니다

설명하기

"replica 3개짜리, 포트 5000번 쓰는 Flask 앱을 위한 Chart를 만들어줘"처럼 사람 말로 요청

생성/수정

AI가 Chart.yaml, values.yaml, templates 파일을 실제로 작성하거나 기존 파일을 고쳐줌

검증/설명

"이 에러가 왜 나?"라고 물으면 원인을 찾아 설명하고, 고칠 방법을 제안

더 정확하게 돕는 방법: Helm MCP 서버

AI가 추측 대신, 진짜 데이터를 보고 답하게 만듭니다



방금 방식의 한계

AI는 실제 Helm 저장소에 뭐가 있는지 확인할 방법이 없어서, 존재하지 않는 필드나 예전 버전 문법을 그럴듯하게 지어내는 (hallucination) 경우가 있습니다.



MCP(Model Context Protocol)가 하는 일

AI가 실제 저장소를 조회하는 도구를 직접 호출해서, 진짜 Chart 목록·버전·values.yaml 내용을 확인한 뒤 답하게 합니다. 지어내지 않고 확인 후 답하는 것이 핵심입니다.

실제 오픈소스 도구와 사이트

지금 바로 검색해서 찾아볼 수 있는 것들입니다

이름	하는 일	사이트
mcp-helm	Helm 저장소의 실제 Chart·버전·values.yaml 조회	github.com/zekker6/mcp-helm
helm-chart-mcp	Chart 의존성 분석과 values 설정 검증	github.com/V3RO/helm-chart-mcp
mcp-server-kubernetes	kubectl+Helm+ArgoCD를 함께 다루는 종합 도구	github.com/Flux159/mcp-server-kubernetes
kubernetes-mcp-server	Red Hat이 만든 K8s+OpenShift+Helm 도구	github.com/containers/kubernetes-mcp-server
Artifact Hub	Helm Chart 자체를 검색하는 공개 사이트 (MCP 아님)	artifacthub.io

오늘 소개한 내용, 하나로 모아보면

우리가 배운 각 단계에 AI가 한 겹씩 더해집니다

CI/CD

AI가 코드 리뷰하고 테스트를 대신 만들어 줌

Docker

AI 모델 자체를 이미지로 만들어 배포함 (Ollama, vLLM)

Kubernetes

전문 도구(KServe, Kubeflow, Seldon)가 AI 모델 서빙을 관리함

Helm

AI 어시스턴트가 Chart 작성과 에러 진단을 도와줌

공통점 네 가지 모두 오늘까지 배운 기초(Docker, kubectl, Helm) 위에 AI가 얹히는 구조입니다 — 기초 없이는 이해도, 활용도 어렵습니다.

더 알아보고 싶다면

오늘 소개한 도구들의 공식 자료입니다

주제	찾아볼 곳
AI 코드 리뷰	GitHub Copilot, CodeRabbit 공식 문서
AI 모델 컨테이너화	ollama.com, docs.vllm.ai
K8s AI 모델 서빙	kserve.github.io, kubeflow.org, docs.seldon.ai
Helm + AI 어시스턴트	지금 쓰고 계신 도구(Claude, Copilot 등)에 그대로 물어보기